Cycle-by-Cycle Pipeline Traces (Sections 8.1–8.4)

WM-8 five-stage pipelined washing-machine microcontroller – ELL7282/ELL782 Introduction to Computer Architecture – Assignment 2

1 How the traces were produced

The test benches `tests/t1_hazards.circ` (Sections 8.1–8.3) and `tests/t2_branch_halt.circ` (Section 8.4) contain the unmodified `cpu` and `mmio` sub-circuits, the instruction ROM holding the test program, the data RAM, a power-on reset (first cycle), a cycle counter and the pipeline-monitor outputs of the CPU. These outputs are the PC of each pipeline register and its valid bit, `STALL`, `FLUSH`, `HALTED`, `FWD_A`/`FWD_B`, the write-back port (`WB_WE`, `WB_RD`, `WB_DATA`) and the data-bus write (`D_WE`, `D_ADDR`, `D_WDATA`). One row is one clock cycle: the state after rising edge n. Each instruction cell shows the instruction address and source text, taken from the `PC` field travelling with it. The rows were recorded by cycle-accurate simulation of the test-bench `.circ` netlists (`tools/circsim.py`); the raw rows are in `tests/expected/*.txt`, in the same tab-separated format that Logisim prints with `java -jar logisim-generic-2.7.1.jar tests/t1_hazards.circ -tty table`, so they can be compared directly.

Notation. `08 LD R1,[R2]` = valid entry; ∅ `09 ADD R3,R1` = entry with `V`=0 (bubble or squashed instruction; its stale fields are shown only for orientation). *Stall* = `STALL` (PC and IF/ID hold at the end of the cycle, ID/EX receives a bubble). *Flush* = taken branch/jump or HALT in EX (IF/ID and ID/EX invalidated at the end of the cycle). *Fwd A/B*: source of the EX operand `ra`/`rb` (`-` = value read in ID). *Architectural writes*: register write-back and data-bus stores that commit at the rising edge *ending* the cycle.

2 Section 8.1 – ALU forwarding without stall

Program: `ADD R1,R2 (06); SUB R3,R1 (07)` with R1=01, R2=10, R3=03. In **cycle 10** of the table below, `SUB R3,R1` is in EX while `ADD R1,R2` is in MEM. `FWD_B`=EX/MEM supplies R1=`0x11`, there is no stall, and R3←`0xF2` is written back in cycle 12.

3 Section 8.2 – load-use dependence (exactly one stall)

Program: `LD R1,[R2] (08); ADD R3,R1 (09)`, RAM[0x10]=0x03 written earlier by `ST R3,[R2]`.

Cyc	IF (fetch PC)	ID (IF/ID)	EX (ID/EX)	MEM (EX/MEM)	WB (MEM/WB)	Stall	Flush	Fwd A	Fwd B	Architectural writes (commit at end of cycle)
8	07 SUB R3, R1	06 ADD R1, R2	05 NOP	04 ST R3, [R2]	03 LDI R4, 0x20	0	0	–	–	R4←20; RAM[10]←03
9	08 LD R1, [R2]	07 SUB R3, R1	06 ADD R1, R2	05 NOP	04 ST R3, [R2]	0	0	–	–	–
10	09 ADD R3, R1	08 LD R1, [R2]	07 SUB R3, R1	06 ADD R1, R2	05 NOP	0	0	–	EX/MEM	–
11	0A ADD R1, R2	09 ADD R3, R1	08 LD R1, [R2]	07 SUB R3, R1	06 ADD R1, R2	**1**	0	–	–	R1←11
12	0A ADD R1, R2	09 ADD R3, R1	∅ 09 ADD R3, R1	08 LD R1, [R2]	07 SUB R3, R1	0	0	–	–	R3←F2
13	0B SUB R1, R3	0A ADD R1, R2	09 ADD R3, R1	∅ 09 ADD R3, R1	08 LD R1, [R2]	0	0	–	MEM/WB	R1←03
14	0C ST R1, [R4]	0B SUB R1, R3	0A ADD R1, R2	09 ADD R3, R1	∅ 09 ADD R3, R1	0	0	–	–	–
15	0D LD R5, [R4]	0C ST R1, [R4]	0B SUB R1, R3	0A ADD R1, R2	09 ADD R3, R1	0	0	EX/MEM	MEM/WB	R3←F5

- **Cycle 11**: `LD R1` is in EX and `ADD R3,R1` (which reads R1 via `rb`, `UB`=1) is in ID, so `STALL`=1. This is the only stall cycle.
- **Cycle 12**: the PC (`0x0A`) and IF/ID (`ADD R3,R1`) are frozen and ID/EX holds a bubble (∅ 09). Older instructions advance: `LD` reads RAM in MEM and `SUB` writes R3=`0xF2` in WB. The stalled `ADD` reads that R3 through the WB→ID bypass.
- **Cycle 13**: `ADD R3,R1` is in EX and `LD` is in WB, so `FWD_B`=MEM/WB supplies R1=`0x03`. The load was not forwarded from EX/MEM in cycle 12, when it was not yet ready. **Cycle 15**: R3←`0xF2`+`0x03`=`0xF5`.

4 Section 8.3 – youngest producer forwarded to store data

Program: `ADD R1,R2 (0A); SUB R1,R3 (0B); ST R1,[R4] (0C)` with R1=03, R2=10, R3=F5, R4=20.

Cyc	IF (fetch PC)	ID (IF/ID)	EX (ID/EX)	MEM (EX/MEM)	WB (MEM/WB)	Stall	Flush	Fwd A	Fwd B	Architectural writes (commit at end of cycle)
12	0A ADD R1, R2	09 ADD R3, R1	∅ 09 ADD R3, R1	08 LD R1, [R2]	07 SUB R3, R1	0	0	–	–	R3←F2
13	0B SUB R1, R3	0A ADD R1, R2	09 ADD R3, R1	∅ 09 ADD R3, R1	08 LD R1, [R2]	0	0	–	MEM/WB	R1←03
14	0C ST R1, [R4]	0B SUB R1, R3	0A ADD R1, R2	09 ADD R3, R1	∅ 09 ADD R3, R1	0	0	–	–	–
15	0D LD R5, [R4]	0C ST R1, [R4]	0B SUB R1, R3	0A ADD R1, R2	09 ADD R3, R1	0	0	EX/MEM	MEM/WB	R3←F5
16	0E LDI R7, 0xF5	0D LD R5, [R4]	0C ST R1, [R4]	0B SUB R1, R3	0A ADD R1, R2	0	0	EX/MEM	–	R1←13
17	0F ST R5, [R7]	0E LDI R7, 0xF5	0D LD R5, [R4]	0C ST R1, [R4]	0B SUB R1, R3	0	0	–	–	R1←1E; RAM[20]←1E
18	10 HALT	0F ST R5, [R7]	0E LDI R7, 0xF5	0D LD R5, [R4]	0C ST R1, [R4]	0	0	–	–	–

- **Cycle 15**: `SUB R1,R3` uses both forwarding paths at once: R1 from EX/MEM (`ADD`, `0x13`) and R3 from MEM/WB (`ADD R3,R1`, `0xF5`).
- **Cycle 16**: `ST R1,[R4]` is in EX. *Two* in-flight producers of R1 exist: `SUB` in EX/MEM (`0x1E`) and the older `ADD` in MEM/WB (`0x13`). `FWD_A`=EX/MEM, so the youngest wins and the store data is `0x1E`. The address R4 comes from the register file (`FWD_B`=`-`).
- **Cycle 17**: the store commits once in MEM, RAM[`0x20`]←`0x1E` (not the stale `0x13`). The next instruction `LD R5,[R4]` reads back `0x1E`, which the program then copies to `DISPLAY`.

5 Section 8.4 – taken branch squashes a wrong-path actuator store

Program: `SUB R1,R1 (05); BEQZ R1,target1 (06); ST R2,[R4] (07); ADD R3,R4 (08); target1: (09)` with R2=`0x21` (would open the valve and lock the door) and R4=`0xF4` (`Rvalve` = actuator latch).

Cyc	IF (fetch PC)	ID (IF/ID)	EX (ID/EX)	MEM (EX/MEM)	WB (MEM/WB)	Stall	Flush	Fwd A	Fwd B	Architectural writes (commit at end of cycle)
5	04 LDI R7, 0xF5	03 LDI R1, 0x09	02 LDI R3, 0x07	01 LDI R4, 0xF4	00 LDI R2, 0x21	0	0	–	–	R2←21
6	05 SUB R1, R1	04 LDI R7, 0xF5	03 LDI R1, 0x09	02 LDI R3, 0x07	01 LDI R4, 0xF4	0	0	–	–	R4←F4
7	06 BEQZ R1, target1	05 SUB R1, R1	04 LDI R7, 0xF5	03 LDI R1, 0x09	02 LDI R3, 0x07	0	0	–	–	R3←07
8	07 ST R2, [R4]	06 BEQZ R1, target1	05 SUB R1, R1	04 LDI R7, 0xF5	03 LDI R1, 0x09	0	0	MEM/WB	MEM/WB	R1←09
9	08 ADD R3, R4	07 ST R2, [R4]	06 BEQZ R1, target1	05 SUB R1, R1	04 LDI R7, 0xF5	0	**1**	EX/MEM	–	R7←F5
10	09 BNEZ R1, bad	∅ 08 ADD R3, R4	∅ 07 ST R2, [R4]	06 BEQZ R1, target1	05 SUB R1, R1	0	0	–	–	R1←00
11	0A BNEZ R3, target2	09 BNEZ R1, bad	∅ 08 ADD R3, R4	∅ 07 ST R2, [R4]	06 BEQZ R1, target1	0	0	–	–	–
12	0B HALT	0A BNEZ R3, target2	09 BNEZ R1, bad	∅ 08 ADD R3, R4	∅ 07 ST R2, [R4]	0	0	–	–	–
13	0C ST R2, [R4]	0B HALT	0A BNEZ R3, target2	09 BNEZ R1, bad	∅ 08 ADD R3, R4	0	**1**	–	–	–
14	0D ST R3, [R7]	∅ 0C ST R2, [R4]	∅ 0B HALT	0A BNEZ R3, target2	09 BNEZ R1, bad	0	0	–	–	–

- **Cycle 9**: `BEQZ R1` is in EX and tests R1, forwarded from EX/MEM (`SUB R1,R1` = 0). The branch is taken, so `FLUSH`=1 and `PC`←`0x09`. At the same edge the wrong-path `ST R2,[R4]` (in ID) and `ADD R3,R4` (in IF) are written into ID/EX and IF/ID with `V`=0.
- **Cycles 10–12**: the squashed entries keep moving down the pipe as bubbles. `ST R2,[R4]` reaches MEM in cycle 11 with `V`=0, so `D_WE`=0, the actuator latch stays `0x00` and `VALVE_IN` never rises. `ADD R3,R4` never writes R3. Branch penalty = 2 cycles (09 is fetched in cycle 10).
- **Cycle 12**: `BNEZ R1,bad` is not taken (no flush). **Cycle 13**: `BNEZ R3,target2` is taken and squashes the wrong-path `HALT` at `0x0B`, which therefore never stops the machine (see full trace, Appendix B).

6 Source-use detection (no stalls on fields that are not read)

The decoder emits UA (instruction reads ra=IR[11:9]) and UB (reads rb=IR[8:6]):

UA=1	ADD, SUB, AND, SGEU, ADDI, ST (data), BEQZ, BNEZ (tested register)
UB=1	MOV, ADD, SUB, AND, SGEU, LD (address), ST (address)
neither	NOP, LDI, JMP, HALT, unassigned 0xD/0xE

The load-use check is IFID.V∧IDEX.V∧IDEX.MR∧[(UA∧RA=IDEX.RA) ∨(UB∧RB=IDEX.RA)]. Without UA/UB, the following cases would stall although they read no register: LDI R1,.. after LD R1,.. (ra field equal), JMP 0x4C (bits 8:6 of the target = 001), and ADDI R3,0x40 or BEQZ R2,0x47 (imm bits 7:6 and reserved bit 8 form a fake rb). MOV Rd,Rs and LD Rd,[Rs] do not read Rd, so they must not stall on it. Forwarding selects are qualified the same way (USE_A=IDEX.V∧UA), so the monitor never reports spurious forwarding for bubbles or unused fields. Bubbles (IFID.V=0 or IDEX.V=0) never cause a stall.

7 A store commits at most once

The data-bus write strobe is D_WE=EXMEM.V∧EXMEM.MW, and the RAM, ACT, DISPLAY, T_LIMIT and T_CTRL registers sample it only at the rising edge that ends the MEM cycle. Stalls freeze only PC and IF/ID. EX/MEM and MEM/WB load new contents every cycle, so a store occupies EX/MEM for exactly one cycle and the stalled copy in ID is not yet in MEM. On a flush or halt, the squashed copies are written with V=0. In every trace each store appears in the *writes* column once (e.g. RAM[0x20] in cycle 17 above), and squashed stores never appear (cycle 11 of 8.4).

A Full trace of `t1_hazards` (cycles 0–24)

Cycles 0–1: power-on reset, then the first fetch. Cycle 20: `HALT` in EX flushes the younger `LDI R6` and freezes the PC. From cycle 21 `HALTED`=1, and older instructions drain.

Cyc	IF (fetch PC)	ID (IF/ID)	EX (ID/EX)	MEM (EX/MEM)	WB (MEM/WB)	Stall	Flush	Fwd A	Fwd B	Architectural writes (commit at end of cycle)
0	00 LDI R1, 0x01	∅ 00 LDI R1, 0x01	∅ 00 LDI R1, 0x01	∅ 00 LDI R1, 0x01	∅ 00 LDI R1, 0x01	0	0	–	–	–
1	00 LDI R1, 0x01	∅ 00 LDI R1, 0x01	∅ 00 LDI R1, 0x01	∅ 00 LDI R1, 0x01	∅ 00 LDI R1, 0x01	0	0	–	–	–
2	01 LDI R2, 0x10	00 LDI R1, 0x01	∅ 00 LDI R1, 0x01	∅ 00 LDI R1, 0x01	∅ 00 LDI R1, 0x01	0	0	–	–	–
3	02 LDI R3, 0x03	01 LDI R2, 0x10	00 LDI R1, 0x01	∅ 00 LDI R1, 0x01	∅ 00 LDI R1, 0x01	0	0	–	–	–
4	03 LDI R4, 0x20	02 LDI R3, 0x03	01 LDI R2, 0x10	00 LDI R1, 0x01	∅ 00 LDI R1, 0x01	0	0	–	–	–
5	04 ST R3, [R2]	03 LDI R4, 0x20	02 LDI R3, 0x03	01 LDI R2, 0x10	00 LDI R1, 0x01	0	0	–	–	R1←01
6	05 NOP	04 ST R3, [R2]	03 LDI R4, 0x20	02 LDI R3, 0x03	01 LDI R2, 0x10	0	0	–	–	R2←10
7	06 ADD R1, R2	05 NOP	04 ST R3, [R2]	03 LDI R4, 0x20	02 LDI R3, 0x03	0	0	MEM/WB	–	R3←03
8	07 SUB R3, R1	06 ADD R1, R2	05 NOP	04 ST R3, [R2]	03 LDI R4, 0x20	0	0	–	–	R4←20; RAM[10]←03
9	08 LD R1, [R2]	07 SUB R3, R1	06 ADD R1, R2	05 NOP	04 ST R3, [R2]	0	0	–	–	–
10	09 ADD R3, R1	08 LD R1, [R2]	07 SUB R3, R1	06 ADD R1, R2	05 NOP	0	0	–	EX/MEM	–
11	0A ADD R1, R2	09 ADD R3, R1	08 LD R1, [R2]	07 SUB R3, R1	06 ADD R1, R2	**1**	0	–	–	R1←11
12	0A ADD R1, R2	09 ADD R3, R1	∅ 09 ADD R3, R1	08 LD R1, [R2]	07 SUB R3, R1	0	0	–	–	R3←F2
13	0B SUB R1, R3	0A ADD R1, R2	09 ADD R3, R1	∅ 09 ADD R3, R1	08 LD R1, [R2]	0	0	–	MEM/WB	R1←03
14	0C ST R1, [R4]	0B SUB R1, R3	0A ADD R1, R2	09 ADD R3, R1	∅ 09 ADD R3, R1	0	0	–	–	–
15	0D LD R5, [R4]	0C ST R1, [R4]	0B SUB R1, R3	0A ADD R1, R2	09 ADD R3, R1	0	0	EX/MEM	MEM/WB	R3←F5
16	0E LDI R7, 0xF5	0D LD R5, [R4]	0C ST R1, [R4]	0B SUB R1, R3	0A ADD R1, R2	0	0	EX/MEM	–	R1←13
17	0F ST R5, [R7]	0E LDI R7, 0xF5	0D LD R5, [R4]	0C ST R1, [R4]	0B SUB R1, R3	0	0	–	–	R1←1E; RAM[20]←1E
18	10 HALT	0F ST R5, [R7]	0E LDI R7, 0xF5	0D LD R5, [R4]	0C ST R1, [R4]	0	0	–	–	–
19	11 LDI R6, 0xFF	10 HALT	0F ST R5, [R7]	0E LDI R7, 0xF5	0D LD R5, [R4]	0	0	MEM/WB	EX/MEM	R5←1E
20	12 ST R6, [R7]	11 LDI R6, 0xFF	10 HALT	0F ST R5, [R7]	0E LDI R7, 0xF5	0	**1**	–	–	R7←F5; DISPLAY←1E
21	*halted*	∅ 12 ST R6, [R7]	∅ 11 LDI R6, 0xFF	10 HALT	0F ST R5, [R7]	0	0	–	–	–
22	*halted*	∅ 12 ST R6, [R7]	∅ 12 ST R6, [R7]	∅ 11 LDI R6, 0xFF	10 HALT	0	0	–	–	–
23	*halted*	∅ 12 ST R6, [R7]	∅ 12 ST R6, [R7]	∅ 12 ST R6, [R7]	∅ 11 LDI R6, 0xFF	0	0	–	–	–
24	*halted*	∅ 12 ST R6, [R7]	∅ 12 ST R6, [R7]	∅ 12 ST R6, [R7]	∅ 12 ST R6, [R7]	0	0	–	–	–

B Full trace of `t2_branch_halt` (cycles 0–26)

Wrong-path `HALT`s at `0x0B` (squashed in cycle 13) and `0x0F`/`0x10` (squashed by `JMP` in cycle 17) are ignored. The real `HALT` at `0x13` is in EX in cycle 22 and kills the younger `ST R2,[R4]` at `0x14`.

Cyc	IF (fetch PC)	ID (IF/ID)	EX (ID/EX)	MEM (EX/MEM)	WB (MEM/WB)	Stall	Flush	Fwd A	Fwd B	Architectural writes (commit at end of cycle)
0	00 LDI R2, 0x21	∅ 00 LDI R2, 0x21	∅ 00 LDI R2, 0x21	∅ 00 LDI R2, 0x21	∅ 00 LDI R2, 0x21	0	0	–	–	–
1	00 LDI R2, 0x21	∅ 00 LDI R2, 0x21	∅ 00 LDI R2, 0x21	∅ 00 LDI R2, 0x21	∅ 00 LDI R2, 0x21	0	0	–	–	–
2	01 LDI R4, 0xF4	00 LDI R2, 0x21	∅ 00 LDI R2, 0x21	∅ 00 LDI R2, 0x21	∅ 00 LDI R2, 0x21	0	0	–	–	–
3	02 LDI R3, 0x07	01 LDI R4, 0xF4	00 LDI R2, 0x21	∅ 00 LDI R2, 0x21	∅ 00 LDI R2, 0x21	0	0	–	–	–
4	03 LDI R1, 0x09	02 LDI R3, 0x07	01 LDI R4, 0xF4	00 LDI R2, 0x21	∅ 00 LDI R2, 0x21	0	0	–	–	–
5	04 LDI R7, 0xF5	03 LDI R1, 0x09	02 LDI R3, 0x07	01 LDI R4, 0xF4	00 LDI R2, 0x21	0	0	–	–	R2←21
6	05 SUB R1, R1	04 LDI R7, 0xF5	03 LDI R1, 0x09	02 LDI R3, 0x07	01 LDI R4, 0xF4	0	0	–	–	R4←F4
7	06 BEQZ R1, target1	05 SUB R1, R1	04 LDI R7, 0xF5	03 LDI R1, 0x09	02 LDI R3, 0x07	0	0	–	–	R3←07

Cyc	IF (fetch PC)	ID (IF/ID)	EX (ID/EX)	MEM (EX/MEM)	WB (MEM/WB)	Stall	Flush	Fwd A	Fwd B	Architectural writes (commit at end of cycle)
8	07 ST R2, [R4]	06 BEQZ R1, target1	05 SUB R1, R1	04 LDI R7, 0xF5	03 LDI R1, 0x09	0	0	MEM/WB	MEM/WB	R1←09
9	08 ADD R3, R4	07 ST R2, [R4]	06 BEQZ R1, target1	05 SUB R1, R1	04 LDI R7, 0xF5	0	**1**	EX/MEM	–	R7←F5
10	09 BNEZ R1, bad	∅ 08 ADD R3, R4	∅ 07 ST R2, [R4]	06 BEQZ R1, target1	05 SUB R1, R1	0	0	–	–	R1←00
11	0A BNEZ R3, target2	09 BNEZ R1, bad	∅ 08 ADD R3, R4	∅ 07 ST R2, [R4]	06 BEQZ R1, target1	0	0	–	–	–
12	0B HALT	0A BNEZ R3, target2	09 BNEZ R1, bad	∅ 08 ADD R3, R4	∅ 07 ST R2, [R4]	0	0	–	–	–
13	0C ST R2, [R4]	0B HALT	0A BNEZ R3, target2	09 BNEZ R1, bad	∅ 08 ADD R3, R4	0	**1**	–	–	–
14	0D ST R3, [R7]	∅ 0C ST R2, [R4]	∅ 0B HALT	0A BNEZ R3, target2	09 BNEZ R1, bad	0	0	–	–	–
15	0E JMP target3	0D ST R3, [R7]	∅ 0C ST R2, [R4]	∅ 0B HALT	0A BNEZ R3, target2	0	0	–	–	–
16	0F HALT	0E JMP target3	0D ST R3, [R7]	∅ 0C ST R2, [R4]	∅ 0B HALT	0	0	–	–	–
17	10 HALT	0F HALT	0E JMP target3	0D ST R3, [R7]	∅ 0C ST R2, [R4]	0	**1**	–	–	DISPLAY←07
18	11 LDI R6, 0xAA	∅ 10 HALT	∅ 0F HALT	0E JMP target3	0D ST R3, [R7]	0	0	–	–	–
19	12 ST R6, [R7]	11 LDI R6, 0xAA	∅ 10 HALT	∅ 0F HALT	0E JMP target3	0	0	–	–	–
20	13 HALT	12 ST R6, [R7]	11 LDI R6, 0xAA	∅ 10 HALT	∅ 0F HALT	0	0	–	–	–
21	14 ST R2, [R4]	13 HALT	12 ST R6, [R7]	11 LDI R6, 0xAA	∅ 10 HALT	0	0	EX/MEM	–	–
22	15 LDI R6, 0x55	14 ST R2, [R4]	13 HALT	12 ST R6, [R7]	11 LDI R6, 0xAA	0	**1**	–	–	R6←AA; DISPLAY←AA
23	*halted*	∅ 15 LDI R6, 0x55	∅ 14 ST R2, [R4]	13 HALT	12 ST R6, [R7]	0	0	–	–	–
24	*halted*	∅ 15 LDI R6, 0x55	∅ 15 LDI R6, 0x55	∅ 14 ST R2, [R4]	13 HALT	0	0	–	–	–
25	*halted*	∅ 15 LDI R6, 0x55	∅ 15 LDI R6, 0x55	∅ 15 LDI R6, 0x55	∅ 14 ST R2, [R4]	0	0	–	–	–
26	*halted*	∅ 15 LDI R6, 0x55	∅ 15 LDI R6, 0x55	∅ 15 LDI R6, 0x55	∅ 15 LDI R6, 0x55	0	0	–	–	–

C Test program listings

t1_hazards.asm

Addr	Hex	Fmt	Label	Assembly	Comment
00	1201	I		LDI R1, 0x01	
01	1410	I		LDI R2, 0x10	ALU operand and RAM address
02	1603	I		LDI R3, 0x03	
03	1820	I		LDI R4, 0x20	RAM address for 8.3
04	7680	R		ST R3, [R2]	RAM[0x10] = 0x03 (initialise before the load)
05	0000	N		NOP	
06	3280	R		ADD R1, R2	R1 = 0x11
07	4640	R		SUB R3, R1	R3 = 0x03-0x11 = 0xF2 (R1 from EX/MEM)
08	6280	R		LD R1, [R2]	R1 = RAM[0x10] = 0x03
09	3640	R		ADD R3, R1	1 bubble, R1 from MEM/WB: R3 = 0xF5
0A	3280	R		ADD R1, R2	R1 = 0x13
0B	42C0	R		SUB R1, R3	R1 = 0x13-0xF5 = 0x1E (youngest writer of R1)
0C	7300	R		ST R1, [R4]	RAM[0x20] = 0x1E (EX/MEM beats MEM/WB)
0D	6B00	R		LD R5, [R4]	R5 = 0x1E
0E	1EF5	I		LDI R7, 0xF5	
0F	7BC0	R		ST R5, [R7]	DISPLAY = 0x1E
10	F000	N		HALT	
11	1CFF	I		LDI R6, 0xFF	younger than HALT: never executes
12	7DC0	R		ST R6, [R7]	

t2_branch_halt.asm

Addr	Hex	Fmt	Label	Assembly	Comment
00	1421	I		LDI R2, 0x21	LOCK|VALVE pattern (would open the valve)
01	18F4	I		LDI R4, 0xF4	Rvalve = actuator latch
02	1607	I		LDI R3, 0x07	
03	1209	I		LDI R1, 0x09	
04	1EF5	I		LDI R7, 0xF5	DISPLAY
05	4240	R		SUB R1, R1	R1 = 0
06	9209	I		BEQZ R1, target1	taken, R1 forwarded from EX/MEM
07	7500	R		ST R2, [R4]	wrong path: must not write the valve
08	3700	R		ADD R3, R4	wrong path
09	A216	I	target1	BNEZ R1, bad	not taken (R1 = 0)
0A	A60D	I		BNEZ R3, target2	taken (R3 = 7)
0B	F000	N		HALT	wrong-path HALT: flushed, must not stop
0C	7500	R		ST R2, [R4]	wrong path
0D	77C0	R	target2	ST R3, [R7]	DISPLAY = 0x07 (R3 untouched by wrong path)
0E	B011	I		JMP target3	
0F	F000	N		HALT	wrong-path HALT behind a jump
10	F000	N		HALT	
11	1CAA	I	target3	LDI R6, 0xAA	
12	7DC0	R		ST R6, [R7]	DISPLAY = 0xAA: execution continued
13	F000	N		HALT	real HALT
14	7500	R		ST R2, [R4]	younger than HALT: killed
15	1C55	I		LDI R6, 0x55	
16	1CBD	I	bad	LDI R6, 0xBD	
17	7DC0	R		ST R6, [R7]	
18	F000	N		HALT	